CSE373, MAT373: Analysis of Algorithms, Spring 2021 **Date:** March 23, 2021

Mock Midterm Exam 2

(6:35 PM – 7:50 PM : 75 Minutes)

- This exam will account for 20% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are allowed to *use scratch papers* for your calculations.

Good Luck!

Question	Parts	Points
1. Asymptotic Bounds	$(a), (b), (c)$	$5 + 5 + 5 = 15$
2. Recurrences.	$(a), (b), (c)$	$5 + 5 + 5 = 15$
3. Bidirectional Selection Sort	–	10
4. Partition	–	20
5. Matrix Multiply	$(a), (b)$	$10 + 5 = 15$
Total		75

Question 1. [**15 Points**] **Asymptotic Bounds.** For each of the following statements state whether it is *True* or *False*. You must justify each answer. Assume that all log's are of base 2.

(*a*) [**5 Points**] $n^{\frac{\log \log n}{\log n}} = \Theta(\log n)$

(*b*) [**5 Points**] $2^{cn} = \Omega(2^n)$, where c is a positive constant

(*c*) [**5 Points**] $n^{\sin n + \cos n} = \mathcal{O}\left(n^{\sqrt{2}}\right)$

Question 2. [**15 Points**] **Recurrences.** For each of the functions below either solve it using the Master Theorem or state why the Master Theorem does not apply.

(*a*) [**5 Points**] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 5T\left(\frac{n}{4}\right) + (2 + (-1)^n)\, n^2, & \text{otherwise.} \end{cases}$

(*b*) [**5 Points**] $\frac{1}{2^{\sin^2 n}} T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 2^{\cos^2 n} T\left(\frac{n}{2}\right) + n \log n, & \text{otherwise.} \end{cases}$

(*c*) [**5 Points**] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 5T\left(\frac{n^2}{3}\right) + \Theta(n), & \text{otherwise.} \end{cases}$

Question 3. [10 Points] Bidirectional Selection Sort. The Bidirectional-Selection-Sort algorithm given below accepts an array $A[1..n]$ of n distinct numbers as input, and rearranges the entries of A in increasing order of value. It iterates through the array $\lfloor \frac{n}{2} \rfloor$ times, and in iteration $j \in [1, \lfloor \frac{n}{2} \rfloor]$ it finds the index of the element with the smallest value and the index of the element with the largest value in the subarray $A[j..n-j+1]$. It moves the element with the smallest value to $A[j]$ and the element with the largest value to $A[n-j+1]$ (by correctly swapping elements to make sure that no array element is destroyed in the process) before moving on to the next iteration.

```
BIDIRECTIONAL-SELECTION-SORT ( A )
1.  for j = 1 to ⌊A.length/2⌋ do
2.      // find the index of the entry with the smallest value, and the index
3.      // of the entry with the largest value in A[j..A.length − j + 1]
4.      min = max = j
5.      for i = j + 1 to A.length − j + 1 do
6.          if A[i] < A[min] then min = i
7.          if A[i] > A[max] then max = i
8.      // move A[min] and A[max] to their correct sorted positions
9.      v_min = A[min], v_max = A[max], v_j = A[j], v_{n−j+1} = A[n − j + 1]
10.     rearrange the numbers above in the array locations above such that now
        A[j] = v_min and A[n − j + 1] = v_max
```

Analyze the worst-case running time of Bidirectional-Selection-Sort on an input of size n.

Question 4. [20 Points] Partition. Given an input array $A[p..r]$ of $n = r - p + 1$ numbers, the *Partition* algorithm we saw in the class rearranges the items of A in-place such that for some $q \in [p, r]$ everything in $A[p..q-1]$ is $\leq A[q]$ and everything in $A[q+1..r]$ is $\geq A[q]$.

Recall that the outermost ***for*** loop of the algorithm iterates from $j = p$ to $j = r - 1$.

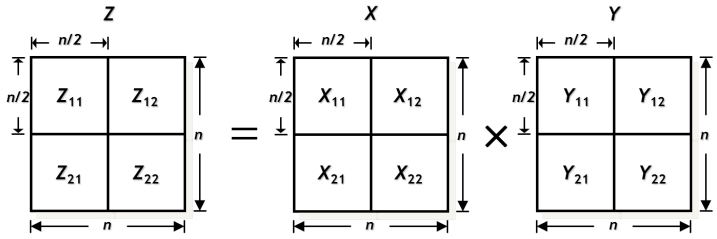
Suppose the following array $A[1..10]$ is given as input to the algorithm. Please show the state of this array after each iteration of the outermost ***for*** loop. Please also show the final state of $A[1..10]$ before the algorithm terminates.

	1	2	3	4	5	6	7	8	9	10
A	14	8	10	5	19	15	2	12	9	11

There must be one line for each value of j. Please start each line with the j value followed by a ":" and then write down the items (separated by commas, no fancy boxes are needed) of the input array in the order they will appear in $A[1..10]$ at the end of the j-th iteration of the loop. The last line should start with the prefix "final state:" and show the final state of the array.

Question 5. [15 Points] Matrix Multiply. The *product* of two $n \times n$ matrices $X[1..n, 1..n]$ and $Y[1..n, 1..n]$ is another $n \times n$ matrix $Z[1..n, 1..n]$ with $Z[i, j] = \sum_{k=1}^{n} X[i, k] \cdot Y[k, j]$ for $1 \leq i, j \leq n$.

The Rec-Matrix-Multiply algorithm given below computes the product of two square matrices using recursive divide-and-conquer. The algorithm accepts two $n \times n$ square matrices $X[1..n, 1..n]$ and $Y[1..n, 1..n]$ as inputs, and recursively generates the product of X and Y in $Z[1..n, 1..n]$. Assume for implicity that $n = 2^r$ for some integer $r \geq 0$. If $n > 1$, X_{11}, X_{12}, X_{21} and X_{22} denote the top-left, top-right, bottom-left and bottom-right quadrants of X, respectively. Similarly for Y and Z. Each quadrant is an $\frac{n}{2} \times \frac{n}{2}$ submatrix of the parent $n \times n$ matrix.




```
Rec-Matrix-Multiply ( X, Y, Z, n )
1.  if n = 1 then
2.      Z = X · Y                                   // a 1 × 1 matrix is just a single number
3.  else
4.      allocate two n × n matrices U and V
5.      // CONQUER:
6.      Rec-Matrix-Multiply ( X11, Y11, U11, n / 2 )   // product of X11 and Y11 is placed in U11
7.      Rec-Matrix-Multiply ( X11, Y12, U12, n / 2 )   // product of X11 and Y12 is placed in U12
8.      Rec-Matrix-Multiply ( X21, Y11, U21, n / 2 )   // product of X21 and Y11 is placed in U21
9.      Rec-Matrix-Multiply ( X21, Y12, U22, n / 2 )   // product of X21 and Y12 is placed in U22
10.     Rec-Matrix-Multiply ( X12, Y21, V11, n / 2 )   // product of X12 and Y21 is placed in V11
11.     Rec-Matrix-Multiply ( X12, Y22, V12, n / 2 )   // product of X12 and Y22 is placed in V12
12.     Rec-Matrix-Multiply ( X22, Y21, V21, n / 2 )   // product of X22 and Y21 is placed in V21
13.     Rec-Matrix-Multiply ( X22, Y22, V22, n / 2 )   // product of X22 and Y22 is placed in V22
14.     // COMBINE: matrix Z is the sum of U and V
15.     for i = 1 to n do
16.       for j = 1 to n do
17.           Z[i, j] = U[i, j] + V[i, j]
```

(*a*) [**10 Points**] Write a recurrence relation describing the running time of Rec-Matrix-Multiply when multiplying two $n \times n$ matrices.

(*b*) [**5 Points**] Solve the recurrence relation from part (*a*).